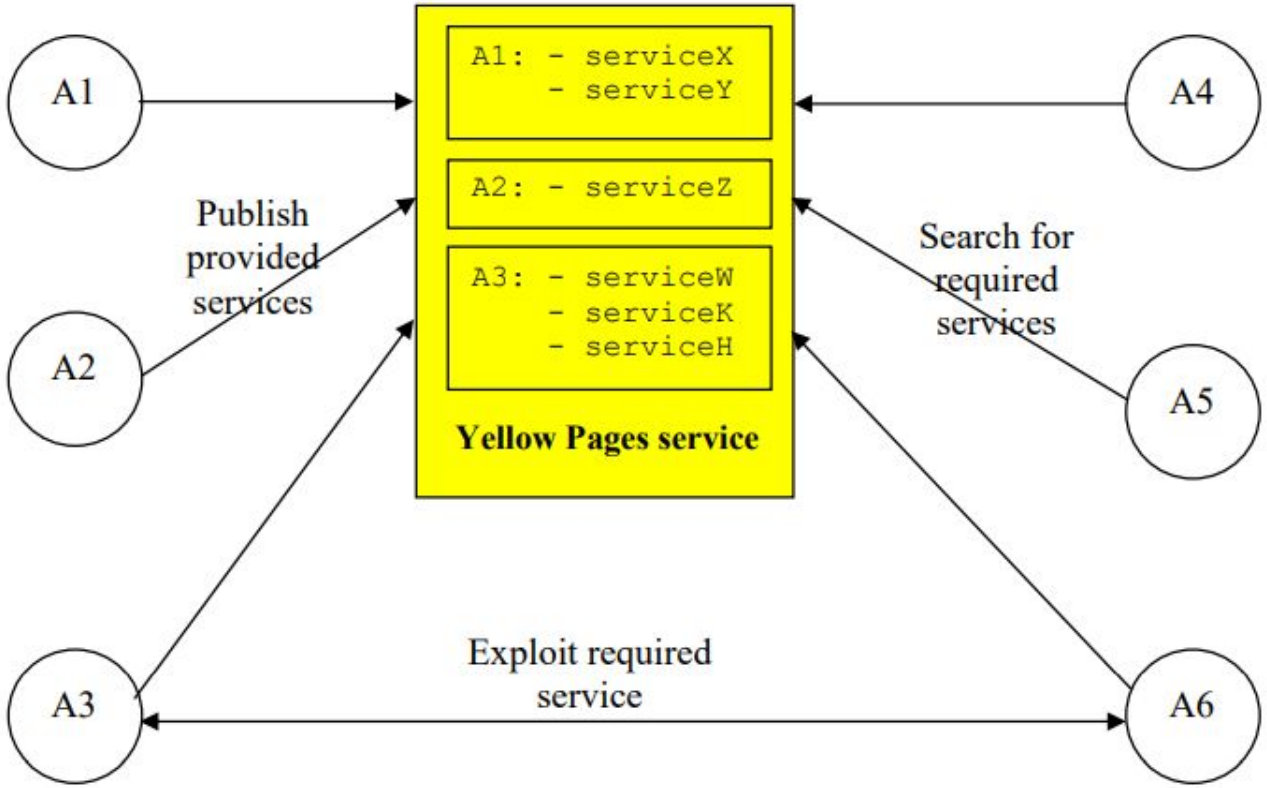
Yellow Page Service

Dr. Kaneeka Vidanage

Dept CS, FOC, KDU

Yellow Page Service

A "yellow pages" service allows agents to publish one or more services they provide so that other agents can find and successively exploit them as described in Figure 4.



Overview

- Yellow Page service is provided by Directory Facilitator [DF] component inside the main container.
- Agents communicate with DF via ACL message template and using SL language [Service Language].
- An agent providing one or more service should provide those info to the DF. Mostly information like : -
    - AID
    - Language used.
    - Ontologies used.
    - Service details.
    - Service specific properties

Should be provided

Service Publishing

Recommended to do the service publishing inside the setup method.

```
protected void setup() {
  ...
  // Register the book-selling service in the yellow page
  DFAgentDescription dfd = new DFAgentDescription();
  dfd.setName(getAID());
  ServiceDescription sd = new ServiceDescription();
  sd.setType("book-selling");
  sd.setName("JADE-book-trading");
  dfd.addServices(sd);
  try {
    DFService.register(this, dfd);
  }
  catch (FIPAException fe) {
    fe.printStackTrace();
  }
  ...
}
```

De-register Services.

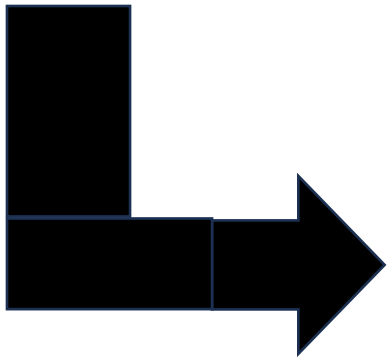
When an agent terminates it is a good practice to de-register published services.

```
protected void takeDown() {
  // Deregister from the yellow pages
  try {
    DFService.deregister(this);
  }
  catch (FIPAException fe) {
    fe.printStackTrace();
  }
  // Close the GUI
  myGui.dispose();
  // Printout a dismissal message
  System.out.println("Seller-agent "+getAID().getName()+" terminating.");
}
```

Searching for Services

An agent wishing to search for services must provide the DF with a template description. The result of the search is the list of all the descriptions that match the provided template. A description matches the template if all the fields specified in the template are present in the description with the same values.

```
public class BookBuyerAgent extends Agent {
  // The title of the book to buy
  private String targetBookTitle;
  // The list of known seller agents
  private AID[] sellerAgents;
```



```
// Put agent initializations here
protected void setup() {
  // Printout a welcome message
  System.out.println("Hello! Buyer-agent "+getAID().getName()+" is ready.");

  // Get the title of the book to buy as a start-up argument
  Object[] args = getArguments();
  if (args != null && args.length > 0) {
    targetBookTitle = (String) args[0];
    System.out.println("Trying to buy "+targetBookTitle);

    // Add a TickerBehaviour that schedules a request to seller agents every minute
    addBehaviour(new TickerBehaviour(this, 60000) {
      protected void onTick() {
        // Update the list of seller agents
        DFAgentDescription template = new DFAgentDescription();
        ServiceDescription sd = new ServiceDescription();
        sd.setType("book-selling");
        template.addServices(sd);
        try {
          DFAgentDescription[] result = DFService.search(myAgent, template);
          sellerAgents = new AID[result.length];
          for (int i = 0; i < result.length; ++i) {
            sellerAgents[i] = result[i].getName();
          }
        }
        catch (FIPAException fe) {
          fe.printStackTrace();
        }
        // Perform the request
        myAgent.addBehaviour(new RequestPerformer());
      }
    } );
```